

Systematic Lean 4 Proof Simplification: MPSLean Refactoring Report

35 files audited, 15 reduced · 7 waves · 658 lines removed (12%) · Lean 4 + Mathlib v4.27.0

5,379 → 4,721 lines · 3 concurrent AI agents · ~3 hours wall-clock · zero regressions

What is MPSLean?

A Lean 4 formalization of:

- Quantum Perron–Frobenius theorem
Evans–Høegh-Krohn [1]
- Matrix Product States (MPS)
Fundamental Theorem [2]
- Wolf’s textbook scaffolding
Quantum Channels & Operations [3]

Stack: Lean 4 + Mathlib (v4.27.0)

By the numbers (post-simplification)

4,721	lines of Lean 4
35	content files
6	subdirectories
3	sorry remaining
1	axiom

The Simplification Campaign

Goal: Systematically audit *all* 35 Lean 4 files; simplify without changing any theorem statements.

Method:

1. 7 waves, each with 3 concurrent AI agents
2. Each agent: specialized `leanSimplifier` tool
(model: `opus46T`, tools: `lean.diagnostics`, `lean.loogle`, etc.)
3. Agents assigned to **different subdirectories** to avoid build cache conflicts
4. Each agent: `edit` → `build` → `verify` → `git commit`

Wave schedule:

Wave	Files (3 per wave)
1	PositiveMap, Transfer, TracePairing
2	KadisonSchwarz, CPPrimitive, Defs
3	CesaroFixedPoint, FundThm, MultiBlock
4	QPF/PosDef, QPF/Uniq., CanonicalForm
5	Irreducible, SpectralGap, MixedTransfer
6	BlockPerm, SkolemNoether, BasisNormal
7	Construction, FTMulti, Remaining

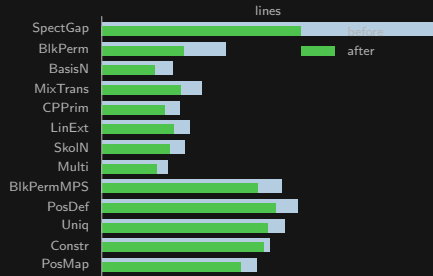
* CesaroFixedPoint grew (+88 lines) due to sorry-isolation fixes.

Results Summary: 5,379 → 4,721 Lines (12% Reduction)

Top reductions:

File	Before	After	%
SpectralGap	605	386	−36%
BlockPermutation	223	147	−34%
BasisNormal	128	96	−25%
MixedTransfer	181	143	−21%
CPPrimitive	140	114	−19%
LinearExtension	159	130	−18%
SkolemNoether	149	122	−18%
MultiBlock	119	100	−16%
BlockPermMPS	325	281	−14%

All files:

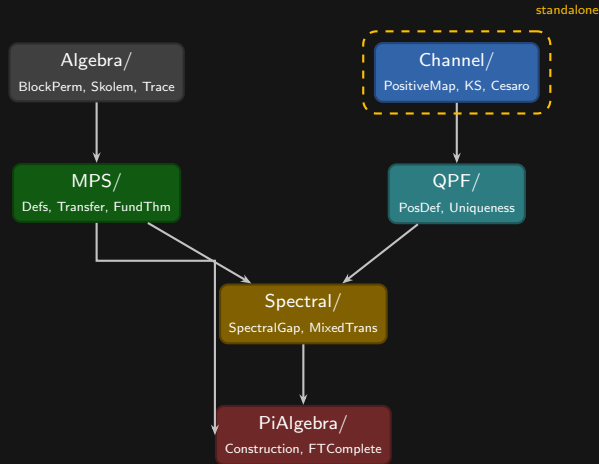


Directory Structure

```
MPSLean/  
  Algebra/          (* 4 files, 403 lines *)  
    BlockPermutation.lean  
    SkolemNoether.lean  
    TraceNondeg.lean  
    TracePairing.lean  
  Channel/          (* 9 files, 974 lines *)  
    PositiveMap.lean  
    KadisonSchwarz.lean  
    Irreducible.lean  
    CesaroFixedPoint.lean  
    Schwarz.lean    (* placeholder *)  
    PeripheralSpectrum.lean  
    Primitive.lean  (* placeholder *)  
    FixedPoints.lean (* placeholder *)  
    Ergodic.lean    (* placeholder *)
```

```
MPS/                (* 10 files, 1221 lines *)  
  Defs.lean  
  Transfer.lean  
  CPPrimitive.lean  
  CanonicalForm.lean  
  LinearExtension.lean  
  FundamentalTheorem.lean  
  FundamentalTheoremMulti.lean  
  MultiBlock.lean  
  BasisNormal.lean  
  BlockPermutationMPS.lean  
QPF/                (* 2 files, 614 lines *)  
  PosDef.lean  
  Uniqueness.lean  
Spectral/           (* 3 files, 643 lines *)  
  SpectralGap.lean  
  MixedTransfer.lean  
  CrossCorrelation.lean  
PiAlgebra/          (* 3 files, 672 lines *)  
  Construction.lean  
  BlockSeparation.lean  
  FundamentalTheoremComplete.lean  
  
  (* Root-level hubs, 194 lines *)  
  QuantumPerronFrobenius.lean  
  PositiveMapSpectral.lean  
  PiAlgebraExtension.lean  
  TransferSpectral.lean  
MPSLean.lean        (* import root *)
```

Dependency Structure



Key insight: **Channel/** is fully standalone (no MPS imports) — reusable for PEPS, MPO, or any quantum channel formalization.

Structural Refactoring (Before Simplification)

4 commits restructured the codebase:

1. **Split god files** — 4 monolithic files (~800+ lines each) split into 35 focused modules
2. **Flatten nesting** — Removed redundant MPSLean/MPS/ nesting
3. **DRY cleanup** — Deduplicate spectral helpers, remove scratch/dead code
4. **Layer separation** — Create Algebra/Channel/MPS/QPF/Spectral/PiAlgebra layers; Channel/ standalone

Before → After:

Before

- 4 files (800+ lines each)
- Flat structure
- Tangled dependencies

After

- 35 files × 6 subdirectories
- Layered architecture
- Channel/ reusable standalone

Pattern 1: Term-Mode Conversion

Replace simple tactic proofs with direct term-mode expressions.

Before:

```
theorem foo (h : A) : B := by
  intro x
  exact bar h x
```

After:

```
theorem foo (h : A) : B :=
  fun x => bar h x
```

Why it helps:

- Eliminates tactic overhead for trivially constructive proofs
- Common in lemmas that just compose hypotheses
- Also: `by exfalso; exact h → absurd ... h`

Pattern 2: Simp Consolidation

Merge scattered `rw`/`simp` sequences into a single call.

Before:

```
rw [map_add]
simp [mul_comm]
rw [trace_transpose]
ring
```

After:

```
simp only [map_add, mul_comm, trace_transpose, ring]
```

Variants found:

- `unfold X; rw [Y]; simp using Z` $\rightarrow$ `simp only [X, Y, Z]`
- `show ... from by rw [...]` $\rightarrow$ `direct simp only`
- `conv` blocks with simple rewrites $\rightarrow$ `simp only` with targeted lemmas

Pattern 3: Have-Chain Compression

Inline single-use have bindings to reduce proof length.

Before:

```
have h1 := foo x
have h2 := bar h1
have h3 : baz := quux h2
exact h3
```

After:

```
exact quux (bar (foo x))
```

Guidelines:

- Only inline haves used exactly once
- Keep named haves that aid readability or are reused
- Chain with dot notation: `h.symm.trans h2` instead of named intermediates

Pattern 4: Dead Code Removal

Remove private lemmas that became unused after other simplifications.

Example from `BlockPermutation.lean`:

Removed (13 lines):

```
private theorem componentMap_map_sub (hσ : ...) (i : Fin K)
  (A B : Matrix (Fin D) (Fin D) ℂ) :
  componentMap T σ i (A - B) =
    componentMap T σ i A - componentMap T σ i B := by
  unfold componentMap
  simp [blockDiag'_sub, ...]
  -- ... 10 more lines of manual proof
```

Why dead: After `componentMap_injective` was simplified to 1 line (using `RingHom.injective`), the manual subtraction lemma was no longer needed.

Total dead code removed: ~76 lines across 6 files.

Pattern 5: Leveraging Mathlib API

The biggest wins came from discovering existing Mathlib lemmas.

Before — `componentMap_injective` (14 lines):

```
theorem componentMap_injective ... :  
  Function.Injective (componentMap T  $\sigma$  i) := by  
  let f := componentMapRingHom h  $\sigma$  i  
  have hker : TwoSidedIdeal.ker (f : Matrix _ _  $\mathbb{C}$   $\rightarrow$ * Matrix _ _  $\mathbb{C}$ ) =  $\perp$  := by  
    rcases eq_bot_or_eq_top (TwoSidedIdeal.ker ...) with h | h  
    · exact h  
    · exfalso  
  have h1 : (1 : Matrix ...)  $\in$  ( $\top$  : TwoSidedIdeal _) := trivial  
  rw [← h] at h1; rw [TwoSidedIdeal.mem_ker] at h1  
  change componentMap T  $\sigma$  i 1 = 0 at h1  
  rw [componentMap_map_one h  $\sigma$  i] at h1; exact one_ne_zero h1  
intro x y hxy  
have hsub : x - y  $\in$  TwoSidedIdeal.ker ... := by ...  
...
```

After (1 line):

```
theorem componentMap_injective ... :  
  Function.Injective (componentMap T  $\sigma$  i) :=  
  (componentMapRingHom h  $\sigma$  i).injective
```

Key: `RingHom.injective` for `IsSimpleRing` does the kernel argument automatically.

Pattern 5 (cont.): API Discovery — isCompact

Before — `densityMatrices_isCompact` (10 lines): from `PositiveMap.lean`

```
theorem densityMatrices_isCompact :  
  IsCompact (densityMatrices D) := by  
  haveI : ProperSpace (Matrix (Fin D) (Fin D) ℂ) := ...  
  apply Metric.isCompact_of_isClosed_isBounded  
  · apply IsClosed.inter isClosed_posSemidef  
    exact isClosed_eq ... continuous_const  
  · apply Bornology.IsBounded.subset  
    (posSemidef_trace_bounded_isBounded 1)  
  intro X ⟨hX_psd, hX_tr⟩  
  exact ⟨hX_psd, by rw [hX_tr]; simp⟩
```

After (3 lines):

```
theorem densityMatrices_isCompact :  
  IsCompact (densityMatrices D) := by  
  haveI : ProperSpace (Matrix (Fin D) (Fin D) ℂ) := ...  
  exact Metric.isCompact_of_isClosed_isBounded  
    (isClosed_posSemidef.inter (isClosed_eq ... continuous_const))  
    ((posSemidef_trace_bounded_isBounded 1).subset  
      fun _ ⟨h1, h2⟩ => ⟨h1, by rw [h2]; simp⟩)
```

Key: Dot notation chains (`.inter`, `.subset`) eliminate `apply/exact` scaffolding.

Pattern 6: Tactic Compression

Lean 4's combinator tactics replace verbose bullet-point proofs.

Before:

```
constructor
· intro x
  simp [mul_comm x]
· intro y
  simp [mul_comm y]
```

After:

```
constructor <;> intro z <;> simp [mul_comm z]
```

Common compressions:

- split; • simp; • simp → split <;> simp
- constructor + bullets → exact ⟨..., ...⟩
- refine ⟨_, ?_⟩; intro x; ... → refine ⟨_, fun x => ?_⟩

Pattern 7: Calc Chain Simplification

Multi-step calc blocks often collapse to a single rw or tactic.

Before:

```
calc ||f x|| ≤ ||f|| * ||x||      := f.le_opNorm x
  _ ≤ 1 * ||x||                  := by nlinarith [hf_bound]
  _ = ||x||                      := one_mul _
```

After:

```
nlinarith [f.le_opNorm x, hf_bound]
```

Before:

```
calc A * B * C = A * (B * C)    := by ring
  _ = A * D          := by rw [hBCD]
  _ = E              := by rw [hAD]
```

After:

```
rw [show A * B * C = A * (B * C) from by ring, hBCD, hAD]
```

Savings: SpectralGap alone had ~12 calc blocks simplified, saving ~80 lines. (from SpectralGap.lean)

Pattern 8: noncomm_ring for Matrix Algebra

Multi-step matrix algebra calc chains $\rightarrow$ noncomm_ring + key rewrites.

Before (3-step calc):

```
calc (1 - P) * A i * P * (P * ((A i) extsuperscriptH * (1 - P)))  
  = (1 - P) * A i * (P * P) * (A i) extsuperscriptH * (1 - P)  
    := by noncomm_ring  
  _ = (1 - P) * A i * P * (A i) extsuperscriptH * (1 - P)  
    := by rw [hPP]  
  _ = (1 - P) * (A i * P * (A i) extsuperscriptH) * (1 - P)  
    := by noncomm_ring
```

After (2 lines):

```
rw [show (1 - P) * A i * P * (P * ((A i) extsuperscriptH * (1 - P)))  
  = (1 - P) * A i * (P * P) * (A i) extsuperscriptH * (1 - P)  
  from by noncomm_ring, hProj.2]  
noncomm_ring
```

Pattern: Use noncomm_ring for associativity/distribution steps, interleave with hypothesis rewrites via show ... from by noncomm_ring.

Impact: Especially effective in CPPrimitive.lean and SpectralGap.lean.

Agent Orchestration: The Wave Model



- Each **wave** runs 3 `leanSimplifier` agents in parallel
- Agents use tools: `lean_diagnostics`, `lean_loogle`, `lean_inspect`
- Model: `opus46T` — takes 15–45 min per file
- Total wall-clock: ~ 3 hours for 35 files audited (15 reduced)
- Each wave waits for all 3 agents to finish before starting next

Lesson 1: Agents Must git commit

Problem: Wave 1

- 3 agents successfully simplified files
- Changes made via `edit_file` on filesystem
- **No git commit issued**
- iCloud sync + concurrent agent builds
→ 4 files' work lost
- Had to redo Wave 1 from scratch

Solution: Mandatory commit

- Added to agent instructions:
"git add & git commit after each file"
- **Zero lost work** after Wave 1 redo
- Bonus: `git diff` provides natural progress tracking per file
- Enables easy rollback if a simplification breaks downstream builds

► **Rule:** Any AI agent editing files *must* commit atomically. Filesystem edits without version control are ephemeral.

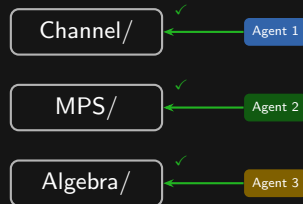
Lesson 2: Subdirectory Isolation

Failure mode:

- Two agents editing files in *same* subdirectory
- Both run `lake build` simultaneously
- Shared `.olean` cache gets corrupted
- Both agents see spurious type errors
- Cascading retries waste 30+ minutes

Solution:

- Max 3 concurrent agents
- Each in a **different** subdirectory:
e.g., `Channel/` + `MPS/` + `Algebra/`
- Serialize agents within same subdir



- Lean's incremental compilation model requires exclusive subdir access.

Campaign Statistics

Effort

Metric	Value
Files audited	35
Files reduced	15
Waves	7
Agents per wave	3
Time per file	15–45 min
Total wall-clock	~3 hours
Agent model	opus46T
Build system	Lake (Lean 4)

Outcome

Metric	Value
Lines before	5,379
Lines after	4,721
Lines removed	658
Reduction	12.2%
Regressions	0
Sorries added	0
Sorries closed	0
Axioms changed	0

- ✓ All 35 files pass lake build with zero new warnings.
- ✓ No theorem statements were changed — only proof terms.

* CesaroFixedPoint grew (+88 lines) due to sorry-isolation fixes; 6 files unchanged.

Remaining Work: 3 Sorries + 1 Axiom

3 sorries remaining:

1. `CesaroFixedPoint.lean`:45

`psd.orthogonal_difference_eq_zero`

Wolf 6.8: PSD orthogonal decomposition argument

2. `SpectralGap.lean`:72

`mixedTransferSpectralRadius.eq_transferMatrix`

Spectral radius identity for mixed transfer

3. `BlockSeparation.lean`:114

`block_powsun_separation`

Vandermonde-based block separation

1 axiom:

- `modulus_one_eigenvalue_implies_gauge`

The hard gauge argument: $|\mu| = 1$ eigenvalue $\implies$ gauge transformation

Channel/ placeholders to fill:

◇ `Schwarz.lean`

Kadison–Schwarz full theory

◇ `PeripheralSpectrum.lean`

Eigenvalues on unit circle

◇ `Primitive.lean`

Primitive channel characterization

◇ `FixedPoints.lean`

Fixed-point algebra

◇ `Ergodic.lean`

Ergodic channels

These are Wolf textbook chapters not yet needed for the MPS fundamental theorem.

Key Takeaways

On AI-assisted proof simplification:

- Parallel AI agents can systematically simplify a **multi-thousand-line** Lean 4 project in hours, not days
- Biggest wins: **Mathlib API discovery** (agents find lemmas humans miss)
- The 8 patterns identified are **universal** — apply to any Lean 4 project
- Zero regressions when each agent independently verifies `lake build`

On orchestration:

- **Commit early, commit often** — filesystem edits are fragile
- **Subdirectory isolation** — Lean's build cache needs exclusive access
- **Wave model** — natural batching respects dependency order
- Diminishing returns: “already-clean” files show $<5\%$ reduction

5,379 → 4,721 lines — cleaner, faster to build, easier to maintain.

References

- [1] David E. Evans and Raphael Høegh-Krohn. Spectral properties of positive maps on C^* -algebras. *Journal of the London Mathematical Society*, s2-17(2):345–355, 1978. doi: 10.1112/jlms/s2-17.2.345.
- [2] D. Pérez-García, F. Verstraete, M. M. Wolf, and J. I. Cirac. Matrix product state representations. *Quantum Information & Computation*, 7(5–6):401–430, 2007. doi: 10.26421/QIC7.5-6-1.
- [3] Michael M. Wolf. Quantum channels & operations: Guided tour. Lecture notes, 2012. URL <https://mediatum.ub.tum.de/doc/1701036/1701036.pdf>. TU Munich.